✓ If each formal parameter, local variable (if any) and return address require one word of storage then the recursion stack space needed by the compiler (which is said to implement tail recursion) for the implementation of the following algorithm is at least: * 2/2

```
Algorithm fib_t (n , curr, prev)
 {
   if (n = = 1)
        return prev;
   return fib_t (n - 1, curr+ prev, curr);
   }
```

- ○ n-1
- ○ n+1
- ◉ c, where c is a constant ✓
- ○ None of the specified options
- ○ n

✓ [SELECT ALL THE CORRECT OPTIONS] For algorithms specified below, which of the following algorithm is faster [requires less time in terms of total number of steps (steps count)]? * 3/3

```
Algorithm Add1(a,b,c,m,n)
{
 for (i=1; i<= m; i++)
     for (j=1; j<= n; j++)
          c[i,j] = a[i,j] + b[i,j];
 }
```

```
Algorithm Add2(a,b,c,m,n)
{
 for (j=1; j<= n; j++)
     for (i=1; i<= m; i++)
          c[i,j] = a[i,j] + b[i,j];
 }
```

- [ ] Add1, if m>n
- [ ] Add2, if m<n
- [x] Add2, if m>n ✓
- [x] Add1, if m<n ✓
- [ ] Other: .....

✓ [SELECT ALL THE CORRECT OPTIONS] Assuming that front and rear are initially initialized to -1, the following algorithms can be used for: * 2/2

```
Algorithm enqueue(item, q, n, rear)
{
                                        // insert item ,using q(0:n-1)
  if (rear = = n-1)
    print("Overflow: the queue is full - insertion is not possible");
  else
   {
    rear = rear+1;
    q[rear] = item;
     }
  return rear;
  }

Algorithm dequeue(q, front, rear)
{
                                    // delete an element ,using q(0:n-1)
  if (front = = rear)
    print("The queue is empty - deletion is not possible");
  else
   {
     front = front - 1;
     item= q[front];
      print("The deleted item is", item);
       }
  return front;
  }
```

- [ ] implementing the insert and delete operations of stack

- [ ] implementing the insert and delete operations of a circular queue
- [x] None of the specified options ✓
- [ ] implementing the insert and delete operations of a non-circular queue

✗ If each formal parameter, local variable (if any) and return address require one word of storage then for the following algorithm the recursion stack space needed is at least: * 0/3

```
Algorithm myfunc(m,n)
{
If (m==0)
        return n+1;
 else if(n==0)
                return myfunc(m-1, 1);
        else
                return myfunc(m-1, myfunc(m,n-1));
  }
```

- ( ) a(2^m + 2^n)+ c, where a and c are constants
- ( ) None of these
- (•) amn+c, where a and c are constants ✗
- ( ) m+n
- ( ) am+c, where a and c are constants

Correct answer

- (•) am+c, where a and c are constants

✓ [SELECT ALL THE CORRECT OPTIONS] Assuming that front and rear are initially initialized to zero, the following algorithms can be used for: * 2/2

```
Algorithm insert (item, q, n, front, rear)
{
                                          // insert item ,using q(0:n-1)
  temp = (rear+1)% n;
  if (front = = rear  )
   print("Overflow: the queue is full - insertion is not possible");
  else
   {
     rear = temp;
     q[rear] = item;
     }
    return rear;
 }

Algorithm delete (q, n, front, rear)
{
                                  // delete an element,using q(0:n-1)
                                 // front and rear are initially initialized to zero
  if (front = = rear)
    print("The queue is empty - deletion is not possible");
  else
   {
     front = (front +1)% n ;
     item= q[front];
      print("The deleted item is", item);
       }
   return front;
  }
```

implementing the insert and delete operations of a non-circular queue

- [ ] implementing the insert and delete operations of a stack
- [ ] implementing the insert and delete operations of a circular queue
- [x] None of the specified options ✓

✓ Let an array be represented as A(2:20, 3:30, 1:25; 0:25, 5:30). Assuming row major representation of the array (using sequential byte addressing scheme) with 2 bytes per element. If k is the address of A(2,3,1,0,5) then the address of the element represented as A(3,10,15,6,12) is * 4/4

- ○ k+3200402
- ◉ k+1202254 ✓
- ○ k+1015612
- ○ k+63
- ○ None of the specified options

Section B

12 of 14 points

There is no part marking. Credits would be awarded only when all the correct options are selected.

✗ *

0/2

If a polynomial is represented in the form

$$(m, e_{m-1}, b_{m-1}, e_{m-2}, b_{m-2}, \ldots, e_o, b_o)$$

where,

the first entry m represents the number of nonzero terms followed by two entries for each term representing an exponent-coefficient pair.

The total number of element locations needed to store the result using the above representation when the polynomial $x^{1000}+4x^{500}+3x^5+2x^4+5x+2$ is added to the polynomial $x^{500}+4x^{200}+9x^4+2x^2+10x+1$ is

- ○ 17
- ○ 8
- ○ None of the specified options
- ○ 10
- ○ 9
- ○ 18
- ◉ 16 ✗

Correct answer

- ◉ 17

✓ [SELECT ALL THE CORRECT OPTIONS] Assuming that front and rear are initially initialized to zero, the following algorithms can be used for: * 2/2

```
Algorithm insert (item, q, n, front, rear)
{
        // insert item, using q(0:n-1)
    temp = (rear+1)% n;
    if (front = = temp )
      print("Overflow: the queue is full - insertion is not possible");
    else
     {
       rear = temp;
       q[rear] = item;
       }
      return rear;
   }
Algorithm delete(q, n, front, rear)
{
         // delete an element, using q(0:n-1)
        // front and rear are initially initialized to zero
    if (front = = rear)
      print("The queue is empty - deletion is not possible");
    else
     {
       front = (front +1)% n ;
       item= q[front];
        print("The deleted item is", item);
         }
```

```
return front;
}
```

- [ ] implementing the insert and delete operations of a non-circular queue
- [ ] implementing the insert and delete operations of a stack
- [x] implementing the insert and delete operations of a circular queue ✓
- [ ] None of the specified options

✓ *

2/2

A general polynomial A(x) can be written as

$$a_n x^n + a_{n-1} x^{n-1} + \ldots + a_1 x + a_0$$

where $a_n \neq 0$ and we say that the degree of A is n.
Then one can represent A(x) as an ordered list of coefficients using a one dimensional array of length $n + 2$,

$$A = (n, a_n, a_{n-1}, \ldots, a_1, a_0)$$

The first element, n is the degree of A followed by the $n + 1$ coefficients in the order of decreasing exponent.

The total number of element locations needed to store the result using the above representation when the polynomial $x^{1000}+4x^{500}+3x^5+2x^4+5x+2$ is added to the polynomial $x^{500}+4x^{200}+9x^4+2x^2+10x+1$ is

- ( ) 1000
- ( ) 17
- ( ) None of the specified options
- ( ) 9
- ( ) 16
- (•) 1002 ✓
- ( ) 1001

8

✓ A sparse matrix containing 500 elements with n non-zero elements (where n << 500) can be stored in a two dimensional array using * 2/2

- n-1 element locations
- 3n element locations
- None of the specified options
- n+1 element locations
- 3n+3 element locations ✓

✓ For the following, the time required in terms of total number of steps (steps count) is * 2/2

```
Algorithm tower(n, source, inter, dest)
{
  if(n>0)
    {
     tower(n-1, source, dest, inter);
     print("Move disk", n, "from", source, "to", dest);
     tower(n-1, inter, source, dest);
    }
}
```

- 2n+c, where c is a constant
- n+1

- ○ n
- ◉ 2^n+c, where c is a constant ✓

✓ [SELECT ALL THE CORRECT OPTIONS] The maximum time [in terms of total number of steps (steps count)] that can be taken by the following algorithm is * 4/4

```
Algorithm polyadd (a, b, c)
{
    p=1; q=1; r=1;
  m=a[0]; n=b[0];
  while(p<=2*m && q<=2*n)
   {
     if(a[p] == b[q])
         {
           c[r+1]=a[p+1]+b[q+1];
           if(c[r+1]!=0)
                 {
                   c[r]=a[p]; r=r+2;
                 }
            p=p+2; q=q+2;
         }
       if(a[p]>b[q])
         {
           c[r+1]=a[p+1];   c[r]=a[p];  p=p+2; r=r+2;
           }
       if(a[p] < b[q])
         {
           c[r+1]=b[q+1];  c[r]=b[q];  q=q+2; r=r+2;
           }
     }
  while(p<=2*m)
    {
      c[r+1]=a[p+1]; c[r]=a[p]; p=p+2; r=r+2;
    }
  while(q<=2*n)
    {
      c[r+1]=b[q+1]; c[r]=b[q]; q=q+2; r=r+2;
    }
  c[0]=(r-1)/2;
}
```



- [ ] None of the specified options
- [ ] bn+c, when m<n, where b and c are constants
- [ ] am+b, when m>n, where a and b are constants
- [x] am+bn+c, where a, b and c are constants ✓

This content is neither created nor endorsed by Google. - Terms of Service - Privacy Policy

Google Forms



School of Computer and Systems Sciences
Jawaharlal Nehru University
CS - 403, Data Structure

Time: 2 hrs. MCA, Mid Sem. Exam. 2022-23 Full marks: 6x5 = 30

Answer any three questions (including subparts) from Section A and Section B is compulsory

Section A

Answer any three questions (including subparts)

1 a) Let an array be represented as A(2:20, 3:30, 1:25, 0:25, 5:30). Assuming row major representation of the array (using sequential byte addressing scheme) with 2 bytes per element. If ***k*** is the address of A(4,5,6,7,5) then the address of the element represented as A(3,10,15,6,12) is _____ [Fill in the blank correctly].

b) If $f(x) = 2+6x+18x^2 + 54x^3 \ldots + \infty$ and $f(x) = O(g(x))$ then $g(x) =$ _____ [Fill in the blank correctly].

(4+2)

2 a) For algorithms specified below, which of the following algorithm is faster [requires less time in terms of total number of steps (steps count)] when:

i) $m > n$ ii) $m < n$

[Explain your answer with suitable justification]

$m > n$.

```
Algorithm Add1(a,b,c,m,n)
{
 for (i=1; i<= m; i++)
    for (j=1; j<= n; j++)
        c[i,j] = a[i,j] + b[i,j];
 }
```

```
Algorithm Add2(a,b,c,m,n)
{
 for (j=1; j<= n; j++)
    for (i=1; i<= m; i++)
        c[i,j] = a[i,j] + b[i,j];
 }
```

b) Let a sparse matrix be stored in an array X(0:t, 0:2) where *t* is the number of nonzero elements. The elements X(0,0) and X(0,1) contain the total the number of rows and the total number of columns of the matrix respectively. X(0,2) contains the total number of nonzero elements of the matrix. Let X(i,0), X(i,1) and X(i,2) contain row number, column number and the value of the nonzero element in the matrix respectively, for $1 <= i <= t$. Write an algorithm (using C pseudocode) to store two sparse matrices (taken as input from the user) in the form (as specified above using X), perform addition of these matrices, store the result in the form (as specified above using X) and display the result (in the form which is usually understood by an user unfamiliar with the representation specified using X). Specify the space and time complexity of your algorithm.

(2+4)

3 a) For the following algorithm (*tower*), derive its time complexity [Show all the steps of your derivation]. The recursion stack space used by this algorithm (*tower*) is at least $2^{n+1} - 1$ [Fill in the blank correctly].

```
Algorithm tower(n, source, inter, dest)
{
  if(n>0)
   {
     tower(n-1, source, dest, inter);
     print("Move disk", n, "from", source, "to", dest);
     tower(n-1, inter, source, dest);
   }
}
```

b) If a polynomial is represented in the form $(m, e_{m-1}, b_{m-1}, e_{m-2}, b_{m-2}, \ldots, e_0, b_0)$ where, the first entry *m* represents the number of non-zero terms followed by two entries (e_i, b_i) for each non-zero term (representing an exponent-coefficient pair) in the order $e_{m-1} > e_{m-2} > \ldots > e_0 >= 0$. The total number of element locations needed to store the result using the above representation when the polynomial $x^{1000}+4x^{500}+3x^5+2x^4+5x+2$ is added to the polynomial $x^{500}+4x^{200}+9x^4+2x^3+10x+1$ is _____ [Fill in the blank correctly].

(4+2)

4 a) Consider the following algorithm

```
Algorithm fib_t (n , curr, prev)
{
  if (n == 1)
      return prev;
  return fib_t (n - 1, curr+ prev, curr);
}
```

If each formal parameter, local variable (if any) and return address require one word of storage then the recursion stack space needed by the compiler (which is said to implement tail recursion) for the implementation of the following algorithm is at least _____. [Select the correct option from the following]

i) n
ii) c, where c is a constant
ii) n+1
iv) n-1

b) Consider the following algorithm

```
Algorithm polyadd (a, b, c)
{
    p=1; q=1; r=1;
  m=a[0]; n=b[0];
  while(p<=2*m && q<=2*n)
   {
     if(a[p] == b[q])
        {
          c[r+1]=a[p+1]+b[q+1];
          if(c[r+1]!=0)
                {
                  c[r]=a[p]; r=r+2;
                }
          p=p+2; q=q+2;
        }
      if(a[p]>b[q])
        {
          c[r+1]=a[p+1];  c[r]=a[p];  p=p+2; r=r+2;
        }
      if(a[p] < b[q])
        {
          c[r+1]=b[q+1]:  c[r]=b[q];  q=q+2; r=r+2;
        }
    }
  while(p<=2*m)
    {
      c[r+1]=a[p+1]; c[r]=a[p]; p=p+2; r=r+2;
    }
  while(q<=2*n)
    {
      c[r+1]=b[q+1]; c[r]=b[q]; q=q+2; r=r+2;
    }
  c[0]=(r-1)/2;
}
```

The worst case time complexity of the algorithm *polyadd* is _____ [Select the correct option from the following]

a) O(m+n) b) O(m) c) O(n) d) O(mn)

(2+4)

Section B

Compulsory

5. Write down algorithms (C pseudocode) to perform the following operations on a circular queue of size ***n*** (implemented using array) depending on users choice (where the value of ***n*** is provided by the user as input) and specify their time complexities using asymptotic notations

i) Insert ***k*** number of elements (provided by the user as input) to the circular queue (where the value of ***k*** is also taken from the user as input)
ii) Delete an element from the circular queue and display the deleted element
iii) Display the contents of the circular queue

(6)

6. Write down algorithms (C pseudocode) to perform the following operations on a stack of size ***n*** (implemented using array) depending on users choice (where the value of ***n*** is provided by the user as input) and specify their time complexities using asymptotic notations

i) Push an element (provided by the user as input) to the stack
ii) Pop ***k*** number of elements from the stack (where the value of ***k*** is taken from the user as input) and print the popped (deleted) elements
iii) Display the contents of the stack

(6)

***

School of Computer and Systems Sciences
Jawaharlal Nehru University
CS - 403, Data Structures

Time: 1 hr. 30 mins. MCA, 1st Sessional Exam. 2024 Full marks: 10x3 = 30

Attempt any three (including subparts)

1 a) What is the difference between direct and indirect recursion ? What is tail recursion and what is its advantage?

b) Write a recursive (but not tail-recursive) algorithm (using *C* pseudocode) to find the factorial of a given number ***n*** (where the value of ***n*** is provided by the user as input) and specify the number stack frames (of the call stack) required for executing the function implementing this algorithm.

c) Write an algorithm (using C pseudocode) to store a list of ***n*** elements taken from the user as input (where the value of ***n*** is taken from the user as input) and perform any of the following operations depending upon user's choice.

i) Insert an element at the kth position in the list (where the value of ***k*** is taken from the user as input)

ii) Sort the list of elements (stored in the array) in ascending order.

(2+3+5)

2 a) Write a tail-recursive algorithm (using ***C*** pseudocode) which takes the value of ***n*** from the user as input and displays the nth term of the Fibonacci series as output. Also specify the number stack frames (of the call stack) required (for the compiler which recognizes tail-recursion and implements it) for executing the function implementing this algorithm.

b) A complex-valued matrix ***X*** is represented by a pair of matrices (A, B), where ***A*** and ***B*** contain real values. Write an algorithm (using ***C*** pseudocode) that computes the product of two complex-valued matrices (A,B) and (C,D), where (A, B) * (C, D) = (A+iB) * (C+iD) = (AC-BD) + i(AD+BC). Determine the number of additions and multiplications if the matrices are all n x n.

c) Explain (by specifying suitable examples of programming languages) **0** - based indexing, **1**-based indexing and **n**-based indexing for arrays.

(3+4+3)

3 a) Write down algorithms (using ***C*** pseudocode) to perform the following operations on a stack (implemented using array) of size ***n*** (where the value of ***n*** is provided by the user as input).

i) Push an element (provided by the user as input) to the stack only if there is no overflow.

ii) Pop ***k*** number of elements (provided by the user as input) from the stack (where the value of ***k*** is also taken from the user as input) only if it is non-empty.

b) Write an algorithm (using *C* pseudocode) to perform the addition of two polynomials [where each polynomial is stored the array as a list of elements in the form (m, e_{m-1}, b_{m-1}, e_{m-2}, b_{m-2}, … , e_0, b_0) where, the first entry m represents the number of non-zero terms followed by two entries (e_i, b_i) for each non-zero term (representing an exponent-coefficient pair) in the order $e_{m-1} > e_{m-2} > \ldots > e_0 >= 0$] and print the resultant polynomial.

(4+6)

4 a) Let an array be represented as A(2:20, 3:30, 1:25, 0:25, 5:30). Assuming row major representation of the array (using sequential byte addressing scheme) with 2 bytes per element. If k is the address of A(4,5,6,7,5) then the address of the element represented as A(5,10,15,6,12) is _____ [Fill in the blank correctly].

b) Write down algorithms (using *C* pseudocode) to perform the following operations on a non-circular queue (implemented using array) of size n (where the value of n is provided by the user as input).

i) Insert k number of elements (provided by the user as input) to the non-circular queue (where the value of k is also taken from the user as input) only if there is no overflow.
ii) Delete an element from the queue only if it is non-empty
iii) Reverse the elements of the non-circular queue using stack

(4+6)

5 a) Let a sparse matrix be stored in an array X(0:t,0:2) where t is the number of nonzero terms. The elements X(0,0) and X(0,1) contain the number of rows and columns of the matrix respectively. X(0,2) contains the number of nonzero terms. Let X(i,0), X(i,1) and X(i,2) contain the row number of the element, the column number of the element and the value of the non-zero element in the matrix respectively, for $1 <= i <= t$. Write an algorithm (using *C* pseudocode) to store two sparse matrices (taken as input from the user) in the form (as specified above using X), perform addition of these matrices, store the result in the form (as specified above using X) and display the result (in the form which is usually understood by an user unfamiliar with the representation specified using X).

b) Specify the output of the following recursive function for the call tower(4, 'A', 'B', 'C');

```
int tower(int n, char source, char inter, char dest)
{
  if(n>0)
    {
     tower(n-1, source, dest, inter);
     printf("\n Move disk %d from %c to %c", n, source, dest);
     tower(n-1, inter, source, dest);
    }
 return 0;
}
```

(5+5)

***

043.

School of Computer and Systems Sciences

Jawaharlal Nehru University, New Delhi

MCA- Sem-I

Subject: Data Structure
Subject Code: CS403

Duration: 1:30 hr
Max Marks: 30

Q1. The integers 1, 2, 3, 4, 5 are pushed onto a stack in that order, and pops may happen at any time between pushes. Which output sequence is impossible? [1]

(a) 3 2 5 4 1 (b) 4 5 3 2 1 (c) 4 3 5 1 2 (d) 1 2 3 4 5

Q2. Two stacks share one array A[0..n-1]. Stack 1 grows from index 0 upward (top1 starts at -1) and Stack 2 grows from index n-1 downward (top2 starts at n). Which condition correctly signals overflow, and why is this design better than splitting the array into two fixed halves? [1]

(a) top1 == n/2 or top2 == n/2; it keeps both stacks equal in size
(b) top1 + 1 == top2; overflow occurs only when the whole array is full
(c) top1 == top2; it saves one comparison per push
(d) top1 + top2 == n; it reduces memory to n/2

Q3. A queue is implemented with a singly linked list. Which design gives O(1) time for both enqueue and dequeue? [1]

(a) Keep only a head pointer; enqueue at the tail, dequeue at the head
(b) Keep head and tail pointers; enqueue at the head, dequeue at the tail
(c) Keep head and tail pointers; enqueue at the tail, dequeue at the head
(d) Keep only a tail pointer; enqueue at the head, dequeue at the tail

Q4. A student writes this function to reverse a singly linked list. What does it actually do for a list of 5 nodes? [1]

```
Node* reverse(Node* head) {
  Node *p = NULL, *q = head, *r;
  while (q != NULL) {
    q->next = p;
    r = q->next;
    p = q;
    q = r;
  }
  return p;
}
```

(a) Reverses the list correctly
(b) Returns the original head as a 1-node list; the other 4 nodes become unreachable (memory leak)
(c) Enters an infinite loop
(d) Dereferences NULL on the first iteration and crashes

Q5. An embedded controller must sort up to 50,000 sensor readings. It has almost no spare RAM and a hard real-time deadline, so the worst case matters more than the average. Stability is not required. Which algorithm is the most defensible choice? [1]

(a) Quicksort, because its average case is fastest
(b) Merge sort, because its worst case is O(n log n)

PTO

(c) Heapsort, because it guarantees O(n log n) in the worst case with O(1) extra space
(d) Insertion sort, because it is simple and in-place

Q6. Suppose every partition in quicksort splits the subarray in a fixed 1 : 9 ratio. What is the running time? [1]

(a) Θ(n^2), because the split is unbalanced
(b) Θ(n log n), because the recursion depth is about log base 10/9 of n and each level costs O(n)
(c) Θ(n log n) only if n is a power of 10
(d) Θ(n), because 90% of the work is skipped at each level

Q7. Which input is most likely to give Insertion Sort a linear running time? [1]

(a) reverse-sorted array (b) A randomly shuffled array
(c) An already sorted array (d) An array with its smallest element at the end

Q8. An engineer wants to sort a large array while minimizing writes to memory, and stability is not required. Which of the three algorithms is generally the best choice? [1]

(a). Bubble Sort, because every pass is linear
(b). Insertion Sort, because it always uses fewer writes
(c). Selection Sort, because it performs at most n−1 swaps
(d). All three perform the same number of writes

Q9.Sort the following elements using Insertion Sort. Show the array after every pass.

25, 12, 18, 6, 30, 10 [5]

Q10. Sort the following elements using Selection Sort. Show the array after each pass.

29, 10, 14, 37, 13, 5 [5]

Q11.Sort the following elements using Heap Sort. Show the max heap after construction and after each extraction.

40, 10, 30, 50, 60, 15, 5 [6]

Q12. Sort the following elements using Quick Sort. Take the first element as the pivot and show each partitioning step.

35, 20, 15, 40, 50, 10, 25 [6]